CLO #2: To analyse complexities of different algorithms using asymptotic notations, complexity classes, and standard complexity function

Question#01: [0.125 for each column * 4 = 0.5/part 0.5*10 = 5 Marks]

Problems	Algorithm Identified	Worst-Case Time Complexity	Space Complexity	Design Technique & Data Structures
Selecting the smallest number of emergency response teams so that all disaster-prone zones are covered				
Determining the maximum revenue obtainable by cutting a metal rod into pieces and selling them based on given prices				
Assigning exam invigilators such that no overlapping exams are supervised by the same person				
Computing the number of possible routes in a transport network using repeated adjacency matrix multiplications				
Detecting whether two flight paths intersect to prevent mid-air collisions				
Matching partially typed system commands with valid commands for fast correction				
Identifying the minimum number of surveillance checkpoints so that every road is monitored				
Finding the number of ways to make a target amount using unlimited coins of given denominations				
Scheduling software module execution while respecting dependency constraints				
Choosing optimal rescue equipment underweight				

constraints to maximize usefulness				

CLO #2: To analyse complexities of different algorithms using asymptotic notations, complexity classes, and standard complexity function

Question#02 (A): [3 Marks] For each of the following questions, indicate whether it is T (True) o r F (False) and justify using some examples e.g. assuming a function

1. $if\ f(n) = O(g(n))\ and\ g(n) = O(h(n)),\ then\ f(n) = \theta(h(n))$

2. $if\ f(n) = nlogn + 5n\ and\ g(n) = nlogn,\ then\ f(n) \in \Omega(g(n))$

3. $if\ f(n) = 2^n\ and\ g(n) = n!,\ then\ f(n) \in O(g(n))$

Question 2 (B): [2 Points] Compute the time complexity of the following. Show all steps clearly

1.
```
for (int i = 1; i <= n; i *= 2)
{
  for (int j = 1; j <= n; j++)
     {
       print("DAA");
     }
 }
```

2.
```
i = n
while i > 1:
    print("FAST")
   i = i // 3
```

Solution:

Problems	Algorithm Identified	Worst-Case Time Complexity	Space Complexity	Design Technique & Data Structures
Selecting the smallest number of emergency response teams so that all disaster-prone zones are covered	Set Cover	$O(2^m \times m)$ (Brute Force) / $O(m \cdot n)$ (Greedy)	$O(m + n)$ / $O(m)$	NP-Complete, Greedy Approximation / Brute Force, Arrays
Determining the maximum revenue obtainable by cutting a metal rod into pieces and selling them based on given prices	Rod Cutting	$O(n^2)$	$O(n)$	Dynamic Programming, 1D Arrays
Assigning exam invigilators such that no overlapping exams are supervised by the same person	Independent Set	$O(2^n)$ (Exact) / $O(n)$ (Approx.)	$O(n)$	NP-Complete, Greedy / Brute Force, Graph using Adjacency List
Computing number of possible routes in a transport network using repeated adjacency matrix multiplications	Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$	Dynamic Programming using 2D Arrays
Detecting whether two flight paths intersect to prevent mid-air collisions	Line Intersection Algorithm	$O(n^2)$	$O(n)$	Computational Geometry, CCW Orientation Test, Arrays / Vectors
Matching partially typed system commands with valid commands for fast correction	Knuth–Morris–Pratt (KMP)	$O(n + m)$	$O(m)$	Pattern Matching using LPS Array
Identifying the minimum number of surveillance checkpoints so that every road is monitored	Vertex Cover	$O(2^n \times n^2)$ (Exact) / $O(n + m)$ (Approx.)	$O(n)$ / $O(n + m)$	NP-Complete, Approximation / Brute Force, Graph with Adjacency List
Finding the number of ways to make a target amount using	Coin Change	$O(n \times$ amount$)$	$O($amount$)$	Dynamic Programming, 1D Arrays

unlimited coins of given denominations				
Scheduling software module execution while respecting dependency constraints	Topological Sort	O(V + E)	O(V)	DFS-based Traversal / Greedy, Directed Graph
Choosing optimal rescue equipment under weight constraints to maximize usefulness	0/1 Knapsack Problem	O(n × W)	O(n × W)	Dynamic Programming using 1D/2D Arrays

Question#02 (A): [3 Marks] For each of the following questions, indicate whether it is T (True) o r F (False) and justify using some examples e.g. assuming a function

1. $if\ f(n) = O(g(n))\ and\ g(n) = O(h(n)),\ then\ f(n) = \theta(h(n))$

   False

   $f(n) = O(g(n)) \rightarrow f(n) \leq c_1 g(n)$ and $g(n) = O(h(n)) \rightarrow g(n) \leq c_2 h(n)$ this implies $f(n) = O(h(n))$

2. $if\ f(n) = nlogn + 5n\ and\ g(n) = nlogn,\ then\ f(n) \in \Omega(g(n))$

   True

   The dominant term in $f(n)$ is $nlogn$. Lower-order terms do not affect asymptotic growth. $nlogn + 5n \geq nlogn$ for large values of n

3. $if\ f(n) = 2^n\ and\ g(n) = n!,\ then\ f(n) \in O(g(n))$

   True

   Factorial grows faster than exponential. $n! = 1 . 2 . 3 \ldots . n > 2^n$ for large n. Hence $2^n \in O(n!)$

Question 2 (B): [2 Points] Compute the time complexity of the following. Show all steps clearly

3.
```
for (int i = 1; i <= n; i *= 2)
{
  for (int j = 1; j <= n; j++)
     {
       print("DAA");
     }
 }
```
Solution: Outer Loop: O(logn) and Inner Loop: O(n). Total O(nlogn)

4.
```
i = n
while i > 1:
    print("FAST")
   i = i // 3
```
Solution: I is divided by 3 each iteration, So number of iterations: $log_3 n$**. Time Complexity: O(logn)**